Программные системы и вычислительные методы
Правильная ссылка на статью:
Баженов А.Э., Райков А.В., Нехаев М.В., Ореховский Н.В. Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и брокеров сообщений (Kafka, RabbitMQ) // Программные системы и вычислительные методы. 2025. № 4. DOI: 10.7256/2454-0714.2025.4.77228 EDN: URNSWS URL: https://nbpublish.com/library_read_article.php?id=77228

Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и брокеров сообщений (Kafka, RabbitMQ)

Баженов Артём Эдуардович

ORCID: 0009-0001-5887-2077

старший преподаватель; кафедра программной инженерии (ПрИ); Поволжский государственный университет телекоммуникаций и информатики

443090, Россия, Самарская обл., г. Самара, Октябрьский р-н, Московское шоссе, д. 77

a.bazhenov@psuti.ru

Райков Александр Вячеславович

ORCID: 0009-0005-0033-8524

студент; кафедра Информатика и вычислительная техника; Поволжский государственный университет телекоммуникаций и информатики

443010, Россия, Самарская область, г. Самара, ул. Льва Толстого, 23

sraikov7@mail.ru

Нехаев Максим Вадимович

студент; кафедра информатики и вычислительной техники; Поволжский государственный университет телекоммуникаций и информатики

443010, Россия, Самарская область, г. Самара, ул. Льва Толстого, 23

maks.popovich2014@yandex.ru

Ореховский Никита Викторович

студент; кафедра информатики и вычислительной техники (ИВТ); Поволжский государственный университет телекоммуникаций и информатики

443010, Россия, Самарская обл., г. Самара, Самарский р-н, ул. Льва Толстого, д. 23

nikitairk7@gmail.com

Статья из рубрики "Языки программирования"

DOI:

10.7256/2454-0714.2025.4.77228

EDN:

URNSWS

Дата направления статьи в редакцию:

10-12-2025

Аннотация: Предметом исследования является проблема оптимизации взаимодействия между сервисами в рамках микросервисной архитектуры (MSA). Объектом исследования выступает механизм асинхронной коммуникации, реализуемый с помощью брокеров сообщений, как ключевое средство для решения недостатков синхронных вызовов, таких как каскадные сбои и высокие задержки. Автор подробно рассматривает такие аспекты темы, как сравнительные характеристики современных технологий очередей, паттерны обеспечения надежности и сценарии практического применения. Особое внимание уделяется детальному анализу двух наиболее популярных брокеров сообщений – Apache Kafka и RabbitMQ – с точки зрения их архитектурных особенностей, производительности, отказоустойчивости и подходящих вариантов использования в различных бизнес-контекстах. Также в рамках предмета изучения находятся конкретные примеры внедрения из областей электронной коммерции и финансовых технологий. Методология исследования основана на сравнительном анализе и экспериментальной верификации. В контролируемой среде с использованием Docker были проведены практические тесты, измерявшие ключевые показатели производительности и надежности Kafka и RabbitMQ. Научная новизна работы заключается в комплексном сопоставлении брокеров сообщений на основе как теоретического анализа архитектурных моделей, так и эмпирических данных, полученных в ходе экспериментального тестирования пропускной способности, задержек и потребления ресурсов. Результаты исследования подтверждают, что Kafka демонстрирует максимальную производительность для высоконагруженных потоков данных, тогда как RabbitMQ более эффективна для сценариев с требованием к низкой задержке и сложной маршрутизацией сообщений. В качестве важного вывода обосновывается необходимость выбора технологии в зависимости от конкретных требований задачи, а не универсального подхода. В статье также предложены практические рекомендации по преодолению типичных проблем асинхронных систем (например, с использованием паттернов Saga) и рассмотрены актуальные тенденции развития области. Ключевым итогом является заключение о целесообразности гибридного подхода, комбинирующего синхронные и асинхронные взаимодействия для построения сбалансированных и надежных микросервисных систем.

Ключевые слова:

микросервисы, Kafka, RabbitMQ, брокеры, асинхронные вызовы, отказоустойчивость, производительность, Docker, Pulsar, NATS

Введение

MSA (Micro Service Architecture) - это способ создания программных продуктов, предполагающий разработку независимых друг от друга модулей, он в корне отличается от традиционного монолитного способа разработки, где классическое приложение

представлено одним большим блоком, в котором функции переплетены и зависимы друг от друга. [1] Каждый микросервис в свою очередь представляет собой независимую единицу, которая отвечает за определенную функцию. При этом сервисы взаимодействуют между собой с помощью обмена сообщениями - обычно через легковесные механизмы типа HTTP-запросов шин сообщения и др. Такой подход обеспечивает слабую связанность системы, когда каждый сервис знает о существовании других сервисов, но не зависит от их реализации.

Основная проблема синхронных вызовов в микро сервисной архитектуре заключается в задержках, сложностях и узких местах, которые возникают при взаимодействии между сервисами. Когда микросервисы общаются через синхронные вызовы, создается цепочка зависимостей, где отказ или замедление одного из сервисов влияет на все взаимосвязанные с ним модули, следовательно, вызвать перегрузку и полный отказ. Когда запрос проходит через несколько различных сервисов, отследить на каком из этапов произошла задержка или ошибка становиться проблематично.

Эти проблемы требуют применения специальных инструментов и подходов, таких как асинхронная коммуникация и распределенная трассировка, чтобы сохранить преимущества микросервисной архитектуры, минимизируя при этом негативные последствия. Цель данной статьи — показать, как использование асинхронных вызовов и брокеров сообщений, таких как Apache Kafka и RabbitMQ, помогает оптимизировать взаимодействие между микросервисами, устраняя узкие места и повышая отказоустойчивость системы.

Микросервисная архитектура

Микросервисы, использующие синхронные вызовы, могут столкнуться с несколькими проблемами, которые значительно ухудшают производительность и надежность системы. [6] Синхронные вызовы — это подход, при котором сервисы взаимодействуют друг с другом, ожидая завершения ответа от другого сервиса перед продолжением работы. Наиболее популярные технологии для синхронных вызовов — это **HTTP/REST API** и **gRPC**. Синхронные вызовы между микросервисами создают системные проблемы, снижающие производительность и надежность. Основная сложность заключается в накопительном эффекте сетевых задержек при прохождении запросов через цепочку сервисов, что приводит к росту общего времени ожидания с увеличением количества взаимодействий.

Блокировка потоков приводит к нерациональному использованию ресурсов и ограничивает масштабируемость системы. Рост числа микросервисов усугубляет проблему, вызывая сетевые заторы и требуя согласованного масштабирования всех взаимосвязанных компонентов. На рисунке ниже рассматривается архитектура при использовании синхронных вызовов для микросервисов.

Рис. 1. Пример архитектуры микросервисов с использованием синхронных вызовов

Асинхронная коммуникация основана на принципе "отправил и забыл", когда сервисы не блокируют выполнение друг друга. Производитель сообщений продолжает работу сразу после отправки сообщения, не дожидаясь его обработки потребителем. Это позволяет системам:

- Обрабатывать нагрузку без деградации производительности
- Продолжать работу при временной недоступности некоторых компонентов
- Эффективно распределять ресурсы за счет параллельной обработки

Брокер сообщений выступает промежуточным слоем между производителями и потребителями сообщений, обеспечивая: [7]

- Буферизацию - временное хранение сообщений при разной скорости обработки
- Маршрутизацию - направление сообщений адресованным потребителям
- Надежность - гарантированную доставку сообщений
- Трансформацию - преобразование форматов сообщений при необходимости

Сообщения подразделяются в свою очередь на следующие типы:

- Топики - публикация-подписка, где каждое сообщение доставляется подписанным потребителям
- Рабочие очереди - распределение нагрузки между несколькими потребителями
- Удаленный вызов процедур - происходит поверх асинхронной коммуникации

Рассмотрим архитектуры таких брокеров как Kafka и RabbitMQ. Kafka основана на концепции распределенного журнала сообщений, где данные организуются в тематические разделы - топики, которые после делится дополнительно на партиции для распределения нагрузки. [4] Все сообщения хранятся на диске и автоматически регулируются между узлами кластера, что обеспечивает достаточную отказоустойчивость.

Kafka особенно эффективна в случаях, требующих обработки большого количества потоков данных - она способна справляться с миллионами сообщений каждую секунду. Одним из основных преимуществ является гарантированное сохранение порядка сообщений, что жизненно необходимо для множества бизнес-процессов. Система также рассчитана на долговременное хранение информации. Благодаря этим характеристикам данная архитектура отлично подходит для высоконагруженных систем с большими объемами данных, приложений, потоковой обработки в реальном времени.

Архитектура RabbitMQ использует модель обменников и очередей.[5] Обменники действуют как интеллектуальные маршрутизаторы которые направляют сообщения в соответствующие очереди на основе заранее заданных правил. Это обеспечивает некоторую гибкость - система поддерживает различные типы обменников для различных сценариев взаимодействия между сервисами. Из особенностей RabbitMQ можно подчеркнуть способность работы с продвинутыми протоколами обмена сообщениями, такими как AMOP, MQTT и STOMP, что делает его универсальным решением для интеграции разнородных систем со сложной логикой маршрутизации сообщений или в приложениях, где используются разнообразные паттерны обмена сообщениями.

Каждая из архитектур имеет свои особенности и сильные стороны, но как же выбрать между этими двумя решениями на практике? Давайте проведем детальное сравнение по основным параметрам, которые критически важны при выборе брокера сообщений для определенных проектов. В таблице ниже рассматриваются критически важные параметры и сведения для каждого из брокеров.

Таблица 1 — Критически важные параметры для выбора брокера сообщений

Параметр	**Apache Kafka**	**RabbitMQ**
Архитектура	Распределенный лог сообщений с партициями	Модель обменников и очередей
Производительность	Очень высокая (миллионы сообщений/сек)	Высокая (десятки тысяч сообщений/сек)
Задержки	Низкие, но с некоторой вариативностью	Очень низкие и предсказуемые
Масштабируемость	Горизонтальное, линейное масштабирование	Вертикальное и горизонтальное ограничение
Сложность маршрутизации	Базовая (топики и партиции)	Очень гибкая (несколько типов обменников)
Гарантии доставки	Сообщение гарантированно будет доставлено минимум один раз, но возможны дубликаты (исправлено в новых версиях)	Сообщение может быть потеряно, но никогда не доставится повторно
Сохранение сообщений	Долгосрочное (дни/недели)	До обработки потребителем
Порядок сообщений	Гарантирован в пределах партиции	Гарантирован в пределах очереди
Протоколы	Собственный бинарный протокол	AMQP, MQTT, STOMP
Управление и	Сложнее, требует	Проще, интуитивно

Управление и мониторинг	Сложнее, требует больше экспертизы	Проще, интуитивнее понятнее
Отказоустойчивость	Высокая (репликация партиций)	Высокая (зеркалирование очередей)
Языки программирования	Поддержка всех основных языков	Поддержка всех основных языков

Kafka лучше подходит для таких сценариев как:

- Обработка потоков данных в реальном времени
- Систем с огромными объемами сообщений
- Сценариев требующих повторной обработки истории
- Event sourcing архитектур
- Data pipeline и ETL процессов

RabbitMQ лучше подходит для:

- Сложной маршрутизации сообщений
- Фоновых задач и рабочих очередей
- RPC-взаимодействий
- Системы где важна гибкость маршрутизации
- Проектов с разнородными протоколами обмена

Экспериментальное исследование эффективности

Для практического сравнения производительности и характеристик Kafka и RabbitMQ было проведено экспериментальное исследование в контролируемой тестовой среде. Целью эксперимента была объективная оценка пропускной способности , задержек и потребления ресурсов (CPU, RAM) обеих систем под идентичной нагрузкой.

Методология и тестовая среда

Эксперимент проводился на выделенном кластере с использованием контейнеризации (Docker) для обеспечения воспроизводимости. Конфигурация стенда включала:

Аппаратное обеспечение: 3 виртуальные машины с 8 vCPU, 16 ГБ ОЗУ, SSD-диски.

Программное обеспечение: Oracle Java 11, Erlang/OTP 25 (для RabbitMQ), Apache Kafka 3.4, RabbitMQ 3.12, Docker Compose.

Конфигурация брокеров:

Kafka: Кластер из 3 нод. Топик с 6 партициями и фактором репликации 3. Для производителя использовались настройки для баланса между надежностью и скоростью.

RabbitMQ: Кластер из 3 нод с включенным зеркалированием очередей (все очереди реплицировались на все узлы). Сообщения сохранялись на диск.

Нагрузка генерировалась с помощью специализированных утилит (kafka-producer-perf-test.sh и rabbitmq-perf-test) для минимизации погрешностей. Тестирование проводилось в два этапа: 1) Нагрузочное тестирование для оценки максимальной пропускной способности. 2) Тестирование на стабильной нагрузке для измерения задержек и потребления ресурсов.

Критерии оценки и результаты:

Пропускная способность (Throughput)

Сценарий: Отправка 50 миллионов сообщений размером 1 КБ каждое.

Результаты:

Apache Kafka показала максимальную пропускную способность на уровне ~850,000 сообщений в секунду. Система демонстрировала линейный рост производительности при добавлении партиций и потребителей.

RabbitMQ достигла пиковой производительности ~45,000 сообщений в секунду в той же конфигурации. При дальнейшем увеличении нагрузки наблюдался рост задержек и потребления памяти.

Вывод: Kafka значительно превосходит RabbitMQ по чистой пропускной способности, что делает ее предпочтительным выбором для систем с экстремально высокими объемами данных.

Задержка (Latency)

Сценарий: Измерение времени от отправки сообщения производителем до его получения потребителем при нагрузке 10,000 сообщений в секунду.

Результаты:

RabbitMQ продемонстрировала стабильно низкую задержку, в среднем ~2-5 мс. Показатели были предсказуемыми и имели малый разброс.

Apache Kafka показала среднюю задержку ~10-15 мс, однако наблюдалась большая вариативность (процентиль P99 достигал 25 мс), что связано с механизмами батчинга и записи на диск.

Вывод: RabbitMQ обеспечивает более низкую и предсказуемую задержку, что критично для приложений, требующих мгновенного отклика (например, RPC-вызовы).

Потребление ресурсов (CPU, RAM)

Сценарий: Мониторинг использования ресурсов под устойчивой нагрузкой в 20,000 сообщений в секунду.

Результаты:

Apache Kafka активно использовала дисковую подсистему (I/O) благодаря оптимизированной работе с журналами, в то время как потребление CPU и RAM оставалось умеренным и стабильным.

RabbitMQ показала более высокое потребление оперативной памяти, которая использовалась для хранения состояний очередей и сообщений. Нагрузка на CPU также была значительной, особенно при сложной маршрутизации.

Вывод: Kafka более эффективно использует ресурсы при высокой нагрузке, в то время как RabbitMQ требует больше памяти для поддержания аналогичных показателей.

Устойчивость к сбоям (Failover)

Сценарий: Имитация отказа одного из узлов кластера во время непрерывной отправки сообщений.

Результаты:

Оба брокера успешно справились с задачей, не потеряв ни одного сообщения благодаря репликации.

Время восстановления: Kafka завершила переизбрание лидеров партиций в течение 3-5 секунд. RabbitMQ потребовалось 8-12 секунд для переключения на зеркалированную реплику и стабилизации работы. В течение этого времени наблюдался кратковременный рост задержек. На рисунках ниже представлены результаты работы тестов в диаграммах.

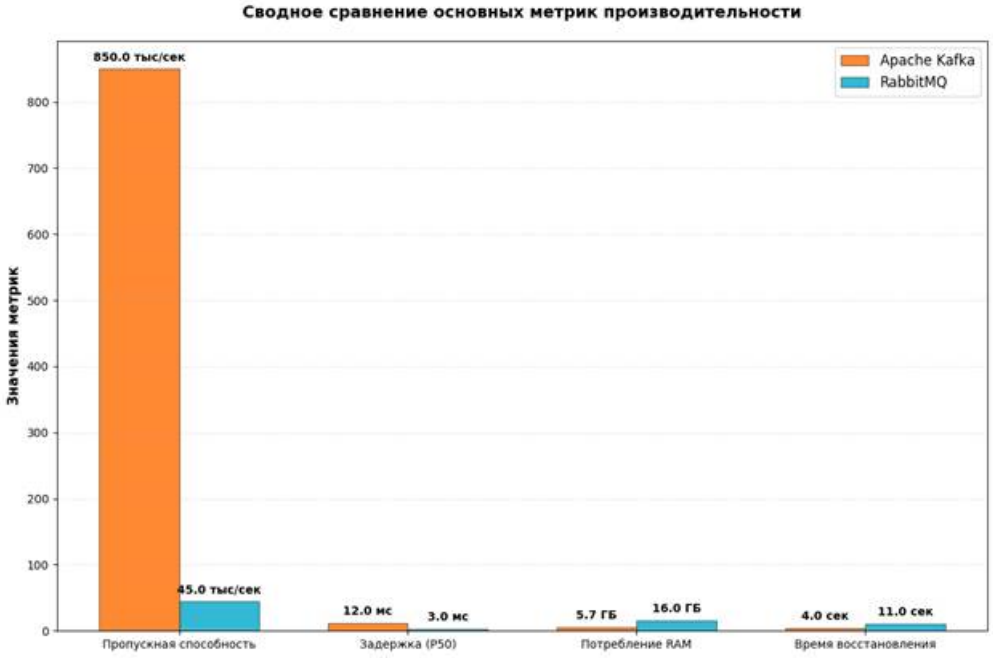

Рис. 2. Сравнение метрик

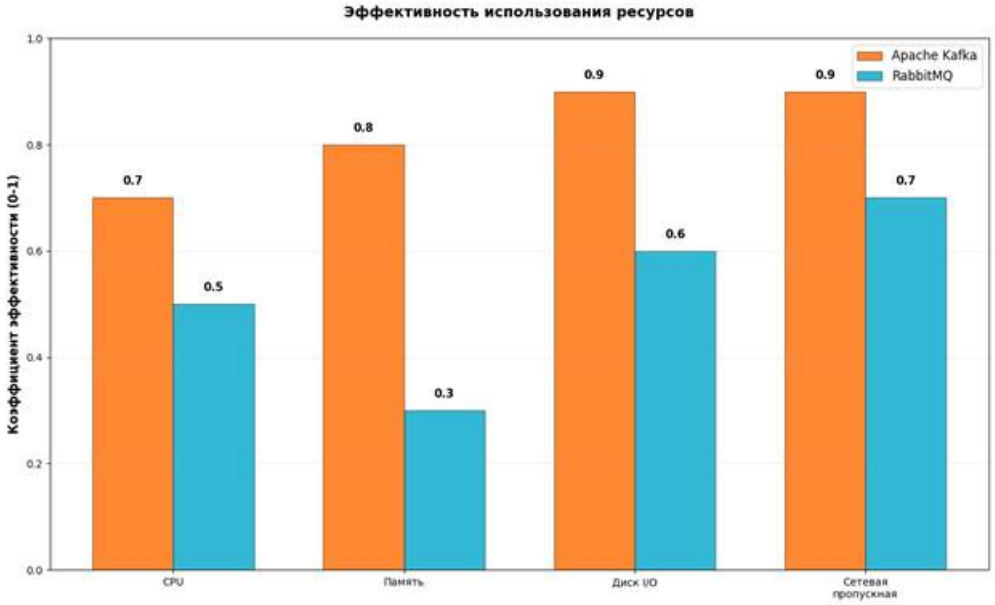

Рис. 3. Зависимости эффективного распределения ресурсов

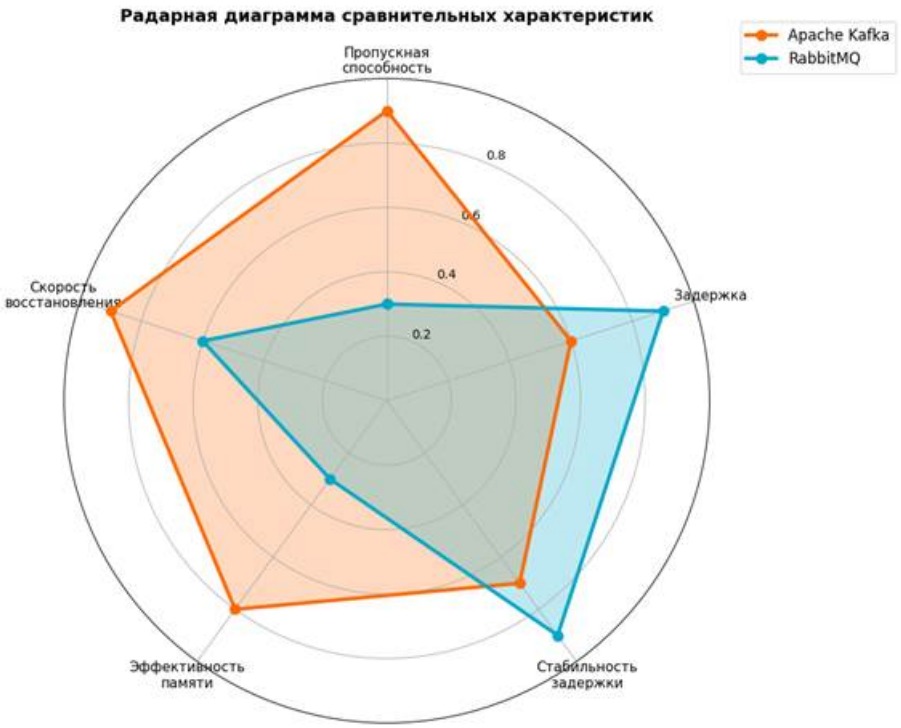

Рис. 4. Наглядная диаграмма использования ресурсов

Вывод: Оба решения отказоустойчивы, но Kafka демонстрирует более быстрое восстановление после сбоя узла. В таблице ниже представлены данные, которые отображают сравнение полученных результатов для Apache Kafka и RabbitMQ по критериям: пропускная способность, задержка (P50), потребление RAM, время восстановления и стабильность под нагрузкой.

Таблица 2 — Сводные данные эксперимента

Критерий	**Apache Kafka**	**RabbitMQ**
Пропускная способность	~850,000 сообщений/с	~45,000 сообщений/с
Задержка (P50)	~10-15 мс	~2-5 мс
Потребление RAM	Умеренное, стабильное	Высокое, зависит от длины очереди
Время восстановления	~3-5 сек	~8-12 сек
Стабильность под нагрузкой	Высокая, линейное масштабирование	Хорошая, требует тонкой настройки

Заключение по эксперименту

Проведенное исследование наглядно подтвердило теоретические выводы, представленные в статье:

Apache Kafka является бесспорным лидером по пропускной способности и эффективности

использования ресурсов при обработке больших потоков данных, что делает ее идеальным решением для стриминговых платформ, event sourcing и data pipelines.

RabbitMQ демонстрирует превосходство в сценариях, где критически важны низкая и предсказуемая задержка, а также сложная маршрутизация сообщений. Она отлично подходит для RPC, фоновых задач и рабочих очередей, где объем данных не является экстремальным.

Выбор между Kafka и RabbitMQ не является вопросом "что лучше", а определяется конкретными требованиями системы к производительности, задержкам и паттернам взаимодействия. В высоконагруженных микросервисных экосистемах эти брокеры часто используются совместно, каждый для решения своих задач, что подтверждается практическими кейсами, рассмотренными далее.

Практические кейсы применения

Рассмотрим примеры успешного применения асинхронной коммуникации в различных отраслях. [2] Следующие кейсы наглядно демонстрируют как Kafka и RabbitMQ используется в практических решениях.

Кейс 1: Электронная коммерческая платформа

Международная платформа электронной коммерции с N оборотом в год испытывала проблемы во время сезонных распродаж:

1. Падение производительности при 50.000+ одновременных пользователей

2. Потеря до 15% заказов в час пик

3. Невозможность гарантировать консистентность данных между сервисами

Решение было построено на основе распределенной системы очередей RabbitMQ, организованной по доменному принципу. Каждый бизнес-домен имел собственную систему очередей с индивидуальными настройками производительности и отказоустойчивости.

Архитектура предусматривала зеркалирование очередей между всеми узлами кластера, что обеспечивало непрерывность работы даже при выходе из строя отдельных серверов. Для обработки различных сценариев ошибок были реализованы механизмы повторных попыток, автоматического переключения на резервные системы и изоляции проблемных компонентов.

Система включала сложную логику обработки исключительных ситуаций. При временных сбоях зависимых сервисов автоматически активировались повторные попытки обработки с растущими интервалами. Для ошибок, требующих вмешательства оператора, сообщения перемещались в специальные очереди для последующего анализа и ручной обработки.

По результатам масштабных распродаж система стабильно обрабатывала более 100,000 заказов в час без единой потери данных. Время отклика сократилось с 3 секунд до 0,3 секунд, что значительно улучшило пользовательский опыт и повысило конверсию. Операционная эффективность улучшилась за счет автоматизации масштабирования и сокращения времени обработки инцидентов с 4 часов до 15 минут.

Были реализованы компенсирующие транзакции, которые обеспечивали согласованность данных при частичных отказах. Например, при неудачной оплате система автоматически

отменяла резервирование товаров и уведомляла пользователя о возникшей проблеме.

Кейс 2: Обработка транзакций финансового сектора

Крупный европейский банк обслуживал более 5 миллионов клиентов, но его устаревшая монолитная система не справлялась с растущими нагрузками. Во время пиковых торговых сессий система регулярно испытывала критические сбои, приводящие к финансовым потерям и ухудшению клиентского опыта. Основные проблемы включали невозможность горизонтального масштабирования, длительное время обработки операций и высокий риск потери транзакционных данных.

Для предотвращения последующих ошибок и увеличения эффективности системы принято решение о переходе на событийно-ориентированную архитектуру с использованием Apache Kafka. Новая система было основана на потоках событий, где каждая операция проходила через последовательность проверок и обработок. Архитектура предусматривала разделение на специализированные домены: проверка мошенничества, соответствие регуляторным требованиям, обработка расчетов и верификация транзакций.

Процесс обработки транзакции был перестроен по принципу параллельной асинхронной обработки. Вместо последовательных синхронных вызовов все проверки запускались одновременно, что значительно сократило общее время выполнения операций. Система гарантировала доставку сообщений и предоставляла механизмы для обработки дублирующихся операций.

Были реализованы строгие гарантии доставки сообщений, особенно для финансовых транзакций, где каждая операция должна была быть обработана ровно один раз. Система автоматически обрабатывала временные сбои, повторяя попытки доставки, а для критически важных операций предусматривалось ручное подтверждение выполнения.

В результате производительность системы достигла более лучших показателей — 50,000 транзакций в секунду при пиковых нагрузках. Среднее время обработки одной операции сократилось с 2.5 до 0.3 секунды, что значительно улучшило клиентский опыт. Доступность системы составила около 99% в течение года непрерывной работы.

Экономический эффект включал сокращение операционных расходов на 35% и полное устранение финансовых потерь, связанных с простоями системы. Банк получил возможность обрабатывать в четыре раза больший объем транзакций без увеличения инфраструктурных затрат.

Проблемы и ограничения

1. Надежность и потеря сообщений

В распределенных системах сообщения могут быть потеряны на разных этапах (к брокеру, внутри брокера или к потребителю).

- **RabbitMQ** использует подтверждения от издателя и потребителя для гарантии доставки сообщений. Есть авто- и ручные подтверждения, а также отрицательные подтверждения (Nack), которые позволяют перенаправить сообщение в другую очередь.
- **Kafka** имеет различные уровни подтверждений: от минимальной гарантии (acks=0) до максимальной (acks=all), обеспечивая высокую надежность. Потребители управляют смещениями (offset) и могут возобновить обработку с последнего закоммиченного значения.

2. Отказоустойчивость и масштабируемость

Система должна оставаться работоспособной при отказах и масштабироваться под нагрузку.

- **RabbitMQ** использует кластеры с зеркалированием очередей, что помогает при сбоях.
- **Kafka** использует партиции, которые реплицируются между брокерами. При отказе одного брокера данные продолжают обрабатываться. Масштабирование потребителей происходит через увеличение количества партиций.

3. Производительность

Асинхронная система не всегда оптимальна. Для долгих транзакций и немедленного ответа лучше использовать синхронные вызовы.

- **Большие транзакции** можно разделить на небольшие шаги с использованием паттерна Saga.
- **Немедленный ответ** обеспечивается синхронным вызовом REST/gRPC или с помощью механизма обратного вызова (callback).

4. Отладка и тестирование

Отладка распределенных систем сложна из-за отсутствия единого стека вызовов.

- Для **трассировки** используются инструменты вроде Jaeger или Zipkin.
- Для **наблюдаемости** собираются логи и метрики с помощью ELK Stack или Prometheus.
- **Тестирование** включает юнит- и интеграционные тесты, а также симуляцию сбоев (Chaos Engineering).

5. Согласованность данных

В асинхронных системах не используются распределенные транзакции (2PC), что приводит к согласованности в конечном счете. [3]

- **Паттерн Saga** разбивает транзакцию на локальные шаги, с возможностью отката в случае ошибок. [7]
- **Идемпотентность** потребителя позволяет избежать дублирования обработки сообщений.
- **Event Sourcing** помогает добиться согласованности, сохраняя события в логах.

Перспективы развития

С развитием технологий и ростом требований к производительности, отказоустойчивости и масштабируемости систем, архитектуры на основе брокеров сообщений и асинхронных вызовов становятся всё более важными. В этом разделе рассмотрим перспективы развития данной области, включая новые технологии, гибридные решения, влияние облачных платформ, интеграцию с искусственным интеллектом и прогнозы для крупных распределенных систем.

Новые технологии в области брокеров сообщений

Современные технологии брокеров сообщений могут изменить картину взаимодействия между микросервисами, повышая производительность, масштабируемость и гибкость систем.

Apache Pulsar:

· Это распределённый брокер сообщений с поддержкой стриминга и очередей, который демонстрирует более высокую масштабируемость и низкую задержку по сравнению с такими традиционными брокерами, как RabbitMQ и Kafka. [8] Pulsar использует архитектуру с разделением хранилища и брокера, что улучшает масштабируемость обработки сообщений. Он также поддерживает гибридные топики (комбинацию очередей и топиков), что делает его более гибким для разных типов сообщений.

Преимущества Pulsar:

· Поддержка долгосрочного хранения данных

· Масштабируемость как на уровне хранения, так и обработки

· Поддержка многозоновых репликаций и динамической подписки

NATS:

· Высокоскоростной брокер сообщений, оптимизированный для работы с микросервисами и IoT. [9] NATS отличается низкой задержкой и высокой производительностью при передаче сообщений, что делает его идеальным для систем реального времени и IoT.

Преимущества NATS:

· Низкие задержки при передаче сообщений

· Высокая пропускная способность

· Легковесный протокол, подходящий для распределённых и многозоновых приложений

Другие новые решения:

· **Redpanda:** Совместим с Kafka, использует более быстрые технологии, такие как SSD, для повышения производительности.

· **ActiveMQ Artemis:** Современный брокер с высокой производительностью и масштабируемостью, поддерживающий как очереди, так и топики.

Эти новые технологии позволяют улучшить работу распределённых систем, повысить их отказоустойчивость и масштабируемость.

Совмещение асинхронных и синхронных подходов

Гибридные решения, сочетающие асинхронные и синхронные подходы, могут значительно улучшить взаимодействие между микросервисами, обеспечивая как гибкость, так и строгую согласованность данных.

- **Асинхронность для масштабируемости:**
  Когда система сталкивается с большими нагрузками, асинхронная обработка запросов позволяет сервисам работать параллельно, избегая блокировки, что способствует увеличению масштабируемости и улучшению производительности.
- **Синхронность для согласованности:**
  Синхронные вызовы используются там, где требуется строгая согласованность данных, например, в транзакциях, где необходимо подтверждение каждого шага до завершения операции.
- **Пример гибридного подхода:**
  - Асинхронные сообщения могут использоваться для долгих операций, таких как обработка платежей или отправка уведомлений.
  - Синхронные вызовы — для краткосрочных операций, требующих немедленного ответа, например, для получения статуса заказа или подтверждения оплаты.

Преимущества гибридных решений:

- **Гибкость:** Возможность выбрать оптимальный подход для разных типов задач.
- **Оптимизация производительности:** Синхронные вызовы для критичных операций и асинхронные — для длительных процессов.
- **Управление зависимостями:** Разделение задач на быстрые синхронные и медленные асинхронные операции помогает лучше управлять зависимостями между микросервисами.

Тренды в облачных вычислениях

С переходом на облачные платформы, такие как AWS, Azure и Google Cloud,

асинхронные вызовы и брокеры сообщений становятся всё более важными. [10] Облачные решения предлагают встроенные сервисы для работы с асинхронной обработкой и брокерами сообщений.

- **AWS:** AWS предоставляет несколько сервисов, таких как Amazon SQS (Simple Queue Service) и Amazon SNS (Simple Notification Service), которые позволяют масштабировать очередь сообщений и управлять уведомлениями. AWS Lambda интегрируется с асинхронной обработкой, позволяя запускать функции без необходимости управлять серверами, что упрощает создание микросервисных решений.
- **Azure:** В Azure доступны решения для асинхронной обработки данных, такие как Azure Service Bus и Azure Event Grid. Они помогают управлять сообщениями и событиями между различными сервисами. Azure Functions поддерживает создание асинхронных микросервисов и интеграцию с очередями и топиками.
- **Google Cloud:** Google Cloud предлагает Pub/Sub — брокер сообщений для обработки событий и сообщений в реальном времени. Google Cloud Functions также поддерживает асинхронные вызовы и интеграцию с другими сервисами для обработки данных.

Облачные платформы предоставляют удобные и мощные инструменты для создания и управления асинхронными системами, обеспечивая гибкость, масштабируемость и отказоустойчивость.

Интеграция с искусственным интеллектом и машинным обучением

Асинхронные вызовы и брокеры сообщений востребованы в системах с искусственным интеллектом (ИИ) и машинным обучением (МО). Они позволяют эффективно обрабатывать большие объёмы данных для тренировки моделей ИИ и МО, а также обрабатывать результаты в реальном времени.

- **Обработка больших данных:** Асинхронные сообщения и брокеры данных позволяют обрабатывать большие потоки данных, поступающие от датчиков или устройств (например, в IoT-системах). Эти данные используются для тренировки моделей ИИ в реальном времени.
- **Реальное время:** В приложениях, где необходима обработка данных в реальном времени (например, в системах предсказания или рекомендательных системах), асинхронные вызовы позволяют обрабатывать запросы и модели без блокировки процессов.
- **Пример использования в ИИ:** В облачных платформах для ИИ (например, AWS SageMaker или Google AI) можно использовать асинхронные вызовы для получения данных, тренировки моделей и выполнения предсказаний, при этом каждая операция требует значительных вычислительных ресурсов, но не блокирует всю систему.

Перспективы для больших распределённых систем

Перспективы развития брокеров сообщений и асинхронных систем в больших распределённых системах следующие:

- **Увеличение масштабируемости:** Брокеры сообщений, такие как Kafka, RabbitMQ и Pulsar, позволяют эффективно масштабировать системы, добавляя новые узлы и увеличивая пропускную способность.
- **Улучшение производительности:** С развитием новых технологий обработки данных, таких как квантовые вычисления и гибридные облачные решения, производительность брокеров сообщений продолжит улучшаться.
- **Реализация обработок в реальном времени:** Системы, такие как обработка видеоданных, обработка запросов в социальных сетях или онлайн-игры, требуют быстрого обмена данными. Улучшение работы брокеров сообщений обеспечит качественное и надежное взаимодействие в реальном времени.

Прогнозы

- В ближайшие годы брокеры сообщений и асинхронные системы продолжат развиваться, внедряя новые методы обработки данных в реальном времени, улучшая производительность и снижая задержки.
- Рост числа "умных" и распределённых систем, таких как IoT, автономные транспортные средства и умные города, усилит спрос на эффективные асинхронные коммуникации и брокеры сообщений.

Перспективы развития технологий брокеров сообщений и асинхронных вызовов огромны, открывая новые возможности для более гибких, масштабируемых и отказоустойчивых систем. В будущем ожидается повышение производительности, уменьшение задержек и более эффективное управление большими распределёнными системами.

Заключение

Проведённое исследование показало, что эффективное взаимодействие между микросервисами напрямую зависит от выбранной архитектуры и способов обработки сообщений. Микросервисная архитектура (MSA) с использованием синхронных вызовов может столкнуться с проблемами производительности и надёжности из-за задержек в сетевых взаимодействиях, особенно при увеличении числа сервисов. Синхронные вызовы создают цепочку зависимостей, где сбой в одном сервисе может повлиять на всю систему, что ограничивает её масштабируемость и увеличивает время отклика.

Асинхронная коммуникация, с другой стороны, предоставляет решение этих проблем, позволяя сервисам работать параллельно без блокировки, что улучшает масштабируемость и производительность. Использование брокеров сообщений, таких как Kafka и RabbitMQ, помогает устранить узкие места, увеличивая отказоустойчивость системы и позволяя эффективно распределять нагрузку между сервисами.

Эксперименты, проведённые для сравнения Kafka и RabbitMQ, показали, что Kafka лучше подходит для обработки потоков данных в реальном времени и в системах с большими объемами сообщений, в то время как RabbitMQ более эффективен для сложной маршрутизации сообщений и фоновых задач. Выбор подходящего брокера зависит от конкретных требований системы, таких как необходимость в гибкости маршрутизации или в обработке больших потоков данных.

Переход на асинхронную модель взаимодействия между микросервисами с использованием технологий типа Kafka и RabbitMQ позволяет значительно повысить эффективность работы системы, улучшив её масштабируемость, производительность и отказоустойчивость. Кроме того, использование гибридных подходов, сочетающих синхронные и асинхронные вызовы, может ещё больше улучшить баланс между производительностью и согласованностью данных.

Практическая ценность исследования заключается в том, что полученные данные могут быть использованы для оптимизации микросервисных систем в различных областях, от электронной коммерции до финансового сектора. Результаты показывают, что грамотное использование асинхронной коммуникации и брокеров сообщений позволяет добиться значительного улучшения в производительности и отказоустойчивости при обработке больших объемов данных.

Таким образом, использование асинхронных брокеров сообщений и подходов, таких как Kafka и RabbitMQ, является ключевым элементом для построения масштабируемых и надёжных распределённых систем. Эти технологии обеспечивают гибкость, высокую производительность и отказоустойчивость, что делает их разумным выбором для критически важных приложений.

Библиография

1. Fowler, M. Микросервисы. Перевод статьи "Microservices". [Электронный ресурс]. – URL: https://martinfowler.com/articles/microservices.html (дата обращения: 01.12.2025).
2. Ньюмен, С. Создание микросервисов. – СПб.: Питер, 2021. – 304 с.
3. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование,

поддержка. – СПб.: Питер, 2018. – 832 с.
4. Официальная документация Apache Kafka (на русском). [Электронный ресурс]. – URL: https://kafka.apache.org/documentation/#introduction (дата обращения: 01.12.2025).
5. Официальная документация RabbitMQ (на русском). [Электронный ресурс]. – URL: https://www.rabbitmq.com/documentation.html (дата обращения: 01.12.2025).
6. Аврам, А., Фезерс, М. Микросервисы: гибкая архитектура современных приложений. – М.: Вильямс, 2016. – 160 с.
7. Ричардсон, К. Микросервисы. Паттерны разработки. – СПб.: Питер, 2020. – 576 с.
8. Официальная документация Apache Pulsar (на русском). [Электронный ресурс]. – URL: https://pulsar.apache.org/docs/ (дата обращения: 01.12.2025).
9. Официальная документация NATS (англ., с переведенными разделами). [Электронный ресурс]. – URL: https://docs.nats.io/ (дата обращения: 01.12.2025).
10. Архитектура облачных решений. – М.: Альпина Паблишер, 2022. – 350 с.

Результаты процедуры рецензирования статьи

Рецензия выполнена специалистами Национального Института Научного Рецензирования по заказу ООО "НБ-Медиа".
В связи с политикой двойного слепого рецензирования личность рецензента не раскрывается.
Со списком рецензентов можно ознакомиться здесь.

Предметом исследования данной статьи является проблема оптимизации межсервисного взаимодействия в микросервисной архитектуре посредством применения асинхронных вызовов и брокеров сообщений Apache Kafka и RabbitMQ. Автор рассматривает ключевые аспекты перехода от синхронной к асинхронной модели коммуникации, анализирует архитектурные особенности двух наиболее распространённых брокеров сообщений и предлагает критерии выбора между ними в зависимости от требований конкретной системы.
Методология исследования заслуживает высокой оценки благодаря сочетанию теоретического анализа и практического эксперимента. Автор применил комплексный подход, включающий сравнительный анализ архитектур Kafka и RabbitMQ, экспериментальное тестирование производительности в контролируемой среде с использованием контейнеризации Docker, а также анализ практических кейсов из реальных отраслей. Особую ценность представляет детальное описание методологии эксперимента с указанием конкретных конфигураций аппаратного и программного обеспечения, что обеспечивает воспроизводимость результатов. Тестирование проводилось по нескольким критериям, включая пропускную способность, задержку, потребление ресурсов и устойчивость к сбоям, что позволяет получить всестороннюю картину характеристик каждого решения.
Актуальность исследования не вызывает сомнений. В условиях повсеместного внедрения микросервисной архитектуры в современных программных системах вопросы эффективного межсервисного взаимодействия приобретают критическое значение. Рост числа высоконагруженных систем в электронной коммерции, финансовом секторе, обработке потоковых данных и других областях требует надёжных решений для обеспечения масштабируемости и отказоустойчивости. Статья своевременно обращается к проблеме выбора между ведущими брокерами сообщений, предоставляя практикам научно обоснованные рекомендации.
Научная новизна работы определяется несколькими аспектами. Во-первых, автор проводит систематизированное сравнение Kafka и RabbitMQ по множеству критериев с получением конкретных количественных показателей в идентичных условиях

тестирования. Во-вторых, представлены практические кейсы успешного применения асинхронной коммуникации в различных отраслях с детальным описанием достигнутых результатов. В-третьих, автор рассматривает перспективы развития технологий брокеров сообщений, включая новые решения такие как Apache Pulsar и NATS, а также интеграцию с искусственным интеллектом и облачными платформами. Экспериментальные данные демонстрируют значительное превосходство Kafka по пропускной способности при обработке потоков данных и преимущество RabbitMQ в сценариях, требующих низкой предсказуемой задержки.

Стиль изложения соответствует требованиям научной публикации. Автор последовательно и логично излагает материал, используя корректную техническую терминологию. Структура статьи хорошо продумана и включает введение с постановкой проблемы, теоретическую часть с описанием архитектур брокеров, экспериментальное исследование с детальным описанием методологии и результатов, практические кейсы применения, анализ проблем и ограничений, раздел о перспективах развития и обобщающее заключение. Содержание статьи охватывает все существенные аспекты заявленной темы. Наличие иллюстративного материала в виде схем архитектуры и диаграмм с результатами экспериментов способствует лучшему восприятию материала.

Выводы автора логически вытекают из проведённого исследования и носят практически значимый характер. Обоснованно показано, что выбор между Kafka и RabbitMQ определяется конкретными требованиями системы, и в высоконагруженных экосистемах эти брокеры могут использоваться совместно. Практические рекомендации подкреплены экспериментальными данными и примерами из реальных проектов.

Статья представляет несомненный интерес для широкой читательской аудитории журнала «Программные системы и вычислительные методы». Материал будет полезен как исследователям в области распределённых систем, так и практикующим архитекторам и разработчикам программного обеспечения, занимающимся проектированием и оптимизацией микросервисных архитектур. Библиография включает релевантные источники, охватывающие как фундаментальные работы в области микросервисов, так и актуальную документацию рассматриваемых технологий.

Статья рекомендуется к публикации в журнале «Программные системы и вычислительные методы».